

60-Minuten-Talk | .NET-Entwickler

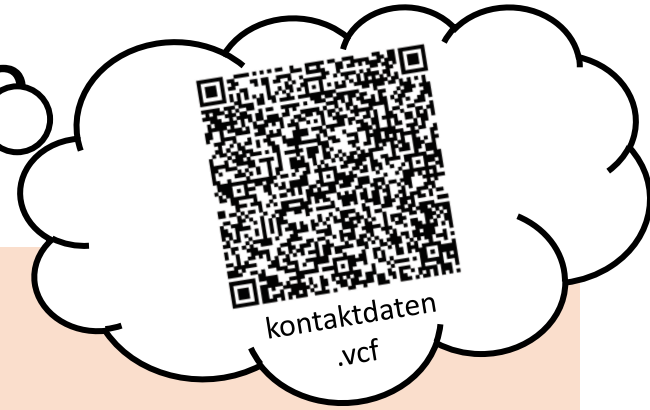
Prompting für .NET-Entwickler

Kollege Copilot

*Du kriegst von GitHub Copilot Mittelmaß statt Excellence?
Dann promptest du falsch.*

Olaf Lischke

Speaker: Olaf Lischke



macht .NET, seit es .NET gibt



versucht, Projekte und Seminare zu kombinieren

Co-Founder und Vorstand KI-Kompass KMU e.V.



singt Tenor in Chören und Musikprojekten



zockte schon auf dem ZX 81, heute ausschließlich auf PC



fotografiert, seit er eine Kamera halten kann

Agenda

01

Copilot verstehen

Modi, Kontext, Grenzen

02

Strategie 1-2

Kontext aufbereiten + Rollen zuweisen

03

Strategie 3-4

Security einbauen + Tasks strukturieren

04

Strategie 5

Typische Fehler und wie du sie vermeidest

05

Copilot steuern

Modi, .md-Dateien, Prompt-Hygiene

Prompting für Entwickler

Copilot verstehen: Die Modi

Ask

Fragen stellen,
Code erklären,
Diskutieren.
Kein Code wird verändert.

Verstehen & Erkunden

VS Code + VS 2026

Plan

Copilot erstellt
einen Schritt-für-Schritt-Plan
für einer Umsetzung.
Kein Code wird verändert.

Komplexe Features

VS Code + VS 2026

Agent

Autonom: Terminal,
Dateien anlegen,
Tests ausführen.

Repetitive Tasks & Edits

VS Code + VS 2026

Tipp:

Erst mit dem Plan-Modus einen Plan erstellen, dann **im gleichen Chat** im Agent-Modus Code generieren lassen.

Achtung: Context-Window beachten, generierten Code reviewen!

Copilot verstehen: Kontext ist alles

Was Copilot sieht

1

Markierter Code

Selektion hat höchste Priorität

2

Chat-Verlauf

Nur der aktuelle Chat-Thread

3

.md-Instruktionen

copilot-instructions.md, prompt-Dateien

4

Offene Dateien

Alle Tabs im Editor können als Kontext zählen

5

Workspace-Index

Semantische Suche im gesamten Repo

Kontext-Qualität

Schlechter Kontext

```
// TODO: fix this
public void Process(object data) {
```

Guter Kontext

```
/// <summary>
/// Verarbeitet eine eingehende Order.
/// Wirft OrderException wenn ungültig.
/// </summary>
public async Task ProcessOrderAsync(
    Order order, CancellationToken ct) {
```

Faustregel:

Benenne Typen, Methoden und Parameter präzise. Schreibe XML-Docs. Halte irrelevante Tabs geschlossen.

Copilot ist so gut wie der Kontext, den du ihm gibst.

Copilot Chat: Referenz-Syntax

Dateien, Code und Kontext gezielt einbinden

Syntax	Bedeutung	Beispiel
#file:<Pfad>	Eine bestimmte Datei einbinden	#file:src/Services/OrderService.cs
#selection	Aktuell markierter Code	Erkläre mir #selection
#editor	Gesamter Inhalt der aktiven Datei	Was macht #editor?
#codebase	Semantische Suche im gesamten Workspace	Finde alle Usages von #codebase
#sym:<Name>	Ein bestimmtes Symbol referenzieren	#sym:OrderProcessor

Tipp: Markierter Code hat höchste Priorität. Irrelevante Tabs schliessen – Copilot ist so gut wie der Kontext, den du ihm gibst.

Copilot Agenten in Visual Studio 2026

Test

Unit Tests generieren, ausführen und Ergebnisse auswerten.

Qualitätssicherung

Debug

Fehler analysieren, Ursachen erklären, Lösungen vorschlagen.

Fehleranalyse

Profiler

Performance-Engpässe finden, Optimierungen empfehlen.

Performance

Modernize

.NET Framework auf .NET 8/9/10 migrieren, Code modernisieren.

Migration

Git

Commit-Messages generieren.
Merge-Konflikte erklären und lösen.
Branch-Strategien vorschlagen.
Diff-Erklärungen.
Pull-Request-Beschreibungen.

Quellcodeablage

Strategie 1 & 2: Kontext + Rolle

S1 Kontext gezielt aufbereiten

Schlecht:

```
// Validiere die Eingabe  
public bool Validate(string input) {
```

Besser:

```
// Validiert eine deutsche IBAN (22 Zeichen, beginnt mit DE).  
// Wirft ArgumentException bei null/leer.  
// Gibt true zurueck wenn gueltig, sonst false.  
public bool Validatelban(string iban) {
```

S2 Rollen zuweisen

Copilot antwortet anders, wenn er eine Rolle hat:

```
"Du bist Senior .NET-Entwickler."  
Fokus: Clean Code, SOLID
```

```
"Du bist Security-Reviewer."  
Fokus: OWASP, Input-Validierung
```

```
"Du bist Performance-Experte."  
Fokus: Alloc, Span<T>, Benchmarks
```

Merksatz: „Context is King“ — aber ohne Rolle regiert er blind.

Strategie 3 & 4: Security + Struktur

S3 Security direkt in den Prompt bauen

Nicht: Erst Code schreiben, dann Security-Review.
Sondern: Security-Anforderungen von Anfang an im Prompt.

```
// Schreibe eine Methode, die SQL-Abfragen  
// gegen eine Produktdatenbank ausführt.  
// Anforderungen:  
// - Nur parametrisierte Queries (kein String-Concat)  
// - Logging bei Fehlern (kein Stack-Trace nach aussen)  
// - Timeout: 30 Sekunden  
public async Task<Product?> GetByIdAsync(int id) {
```

S4 Tasks strukturieren

1. Ziel beschreiben
2. Constraints nennen
3. Format vorgeben
4. Konkret formulieren
5. Iterieren

Je größer der generierte Code-Block, desto wichtiger ist das Review!
Copilot bleibt konsistent im Prompt, aber nicht zwingend über den gesamten Chat-Verlauf!

Strategie 5: Typische Fehler vermeiden



Copilot blind vertrauen

Jeden Vorschlag reviewen. Copilot halluziniert APIs, die nicht existieren.

```
// Copilot schlaegt vor:  
string.IsNullOrEmpty(value).ThrowIfTrue(); // Existiert nicht!
```



Zu vage fragen

Statt "Mach das besser" lieber: "Extrahiere die Validierungslogik in eine eigene Klasse."



Kontext-Reset ignorieren

Langer Chat = verlorener Kontext. Neuen Chat starten und Kontext neu aufbauen.

Die 5 Strategien auf einen Blick

1

Kontext aufbereiten

Präzise Kommentare, offene Dateien und Typen sichtbar machen

2

Rolle zuweisen

"Du bist Senior .NET-Entwickler mit Fokus auf Clean Code."

3

Security einbauen

Sicherheitsanforderungen direkt im Prompt, nicht hinterher

4

Tasks strukturieren

Große Aufgaben in Schritte teilen, iterativ vorgehen

5

Schwächen kennen

Halluzinationen prüfen, vage Prompts vermeiden, Kontext-Reset

Prompt-Anatomie: Das Rezept

ROLLE

Du bist ein erfahrener .NET-Entwickler.

KONTEXT

Wir nutzen .NET 10, EF Core 10, Clean Architecture.

AUFGABE

Schreibe einen Repository für Order mit CRUD.

FORMAT

C#-Klasse; XML-Docs; Guard Clauses statt if-Nesting.

CONSTRAINTS

Keine statischen Methoden. Async/await durchgehend.

Copilot steuern: .md-Dateien & Einstellungen

Profil-Ebene

C:\Users\<Name>\copilot-instructions.md

Gilt für alle Projekte.
Persönliche Präferenzen:
Sprache, Stil, bevorzugte Patterns.

Workspace-Ebene

.github\copilot-instructions.md

Gilt für das gesamte Repo.
Projektregeln: Namenskonventionen,
verbotene Libraries, Architektur.

Prompt-Ebene

.github\prompts*.prompt.md

Wiederverwendbare Prompts
für wiederkehrende Tasks:
z.B. 'unit-test.prompt.md'.

Beispiel copilot-instructions.md:

Schreibe C# 12. Nutze records statt classes für DTOs. Keine statischen Klassen. Alle public APIs mit XML-Docs.

Copilot steuern: Weitere Konfigurationsdateien

AGENTS.md

AGENTS.md (Root oder Unterverzeichnis)

Offener Standard – wird von mehreren KI-Agenten verstanden (Copilot, Claude Code, Codex u.a.).

Gemeinsame Wissensbasis für alle Agenten im Projekt.

Kann im Root oder in Unterverzeichnissen liegen.

Tipp: Parallel zur `copilot-instructions.md` nutzen.

Dateispezifische Anweisungen

`.github/instructions/*.md`

Regeln mit applyTo-Glob-Patterns – gelten nur für bestimmte Dateitypen.

Beispiel: Andere Vorgaben für Controller-Klassen als für Unit-Tests.

Ergänzt die workspace-weite `copilot-instructions.md`.

Agent Skills

`.github/skills/*/SKILL.md`

Aufgabenbezogene Fähigkeiten – werden nur bei passendem Prompt geladen (GA seit Jan 2026).

Beispiele: Docs-Skill, Test-Skill, Security-Skill.

Unterschied zu Instructions: Skills sind bedarfsgesteuert, Instructions immer aktiv.

Setup-Steps

`.github/workflows/copilot-setup-steps.yml`

Konfiguriert die Laufzeitumgebung des Coding Agent.

Legt SDK-Versionen, Dependencies und MCP-Server fest.

Für Self-Hosted Runner unverzichtbar.

Empfohlen: Immer explizit konfigurieren, nicht auf Trial-and-Error verlassen.

Hinweis: AGENTS.md und copilot-instructions.md ergänzen sich. Beide parallel nutzen: copilot-instructions.md für Copilot-spezifische Regeln, AGENTS.md als gemeinsame Basis für alle KI-Agenten im Projekt.

Tipps für die Zukunft

- 1 copilot-instructions.md im nächsten Projekt anlegen
- 2 Einen Prompt mit Rolle + Kontext + Aufgabe schreiben
- 3 Copilot-Vorschläge aktiv reviewen, nicht blind akzeptieren
- 4 Agent-Modus für repetitive Tasks ausprobieren

Copilot ist so gut wie deine Prompts.

Fragen? Diskussion!

Vielen Dank!

Slides und Code-Sample auf

<https://github.com/olaflischke/mddevdays2026/>

